Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory Work No. 2:

Meet the OS You Will Build

(Track B: the xv6 Operating System)

Course: Operating Systems

Elaborated: st. gr. FAF-241 Lungu Ilie

Verified: asist. univ. *Relitman Patricia*

Chişinău – 2026

Contents

1 OBJECTIVE **2**

2 ENVIRONMENT AND SETUP **2**

3 PART 1 – BUILDING AND BOOTING XV6 **2**

4 PART 2 – USING XV6 AS A UNIX **3**
4.1 Listing the Files 3
4.2 Other Programs and Pipes 3
4.3 Observations 4

5 PART 3 – READING THE SOURCE CODE **4**
5.1 Observations 5

6 PART 4 – THE FIRST XV6 PROGRAM **6**
6.1 The Program 6
6.2 Adding the Program to the Build 6
6.3 Testing 7
6.4 The Renamed System Call 7

7 CONCLUSION **7**

1 OBJECTIVE

Over the semester the operating system is not only studied but also grown. The teaching OS is **xv6**: a small but real and bootable Unix, a modern re-implementation of Unix Version 6 of about 9 000 lines of C, running on the RISC-V architecture under the QEMU emulator. In this laboratory work xv6 is built from source, booted, used as the small Unix that it is, its source code is read, and a first program that runs inside it is written and added to the system.

2 ENVIRONMENT AND SETUP

The work is done on a Linux machine with the RISC-V cross-compiler (`riscv64`) and the QEMU emulator installed. The source tree of xv6 is obtained from the course copy or cloned from the official repository:

```
cd ~/xv6-riscv 2>/dev/null || \
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
ls                        # kernel/  user/  Makefile  README ...
```

Listing 1: Obtaining the xv6 source tree.

The first command tries to enter the existing copy and, only if it does not exist, the repository is cloned. The listing shows the main parts of the tree: `kernel/`, `user/`, `mkfs/`, the `Makefile` and the `README`.

3 PART 1 – BUILDING AND BOOTING XV6

```
make qemu
```

Listing 2: Building xv6 and launching it in QEMU.

The `make qemu` target compiles the kernel and all the user programs with the cross-compiler, packs the user programs together with the `README` into a disk image (`fs.img`) and starts QEMU, which boots the kernel from `kernel/kernel` with that image as its disk. After a few seconds the kernel starts and the first user process opens a shell:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

Listing 3: Boot messages of xv6.

The messages `hart 1 starting` and `hart 2 starting` show that the kernel also started two additional processor cores (“harts” in RISC-V terminology); QEMU is configured with three. The `init` process then starts the shell `sh`, which displays the prompt `$`.

To leave xv6 the QEMU exit sequence is used: `Ctrl-A`, release, then `X`. It is a command of the emulator, not a shortcut inside xv6.

4 PART 2 – USING XV6 AS A UNIX

At the xv6 prompt the programs that come with the system were run. They are very similar to the ones used in Laboratory Work 1, because xv6 is an ancestor of Linux in design.

4.1 Listing the Files

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35376
echo           2 4 34312
sleep          2 5 34032
forktest       2 6 17280
grep           2 7 42736
init           2 8 34768
kill           2 9 34240
ln             2 10 34032
ls             2 11 41520
mkdir          2 12 34280
rm             2 13 34264
sh             2 14 56136
stressfs       2 15 35160
usertests      2 16 200056
grind          2 17 50504
wc             2 18 36336
zombie         2 19 33648
logstress      2 20 36184
forphan        2 21 35048
dorphan        2 22 34488
sync           2 23 33712
console        3 24 0
```

Listing 4: Output of `ls` in the root directory of xv6.

The xv6 version of `ls` prints for every entry its name, its type (1 – directory, 2 – file, 3 – device), its inode number and its size in bytes. The list contains the `README` and all the programs of the system, including the program `sleep` written in Part 4, and the device file `console` – the same "everything is a file" idea seen in Laboratory Work 1.

4.2 Other Programs and Pipes

```
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
[...]
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 35376
echo           2 4 34312
wc             2 18 36336
sync           2 23 33712
console        3 24 0
$ wc README
```

```
48 336 2441 README
```

Listing 5: Running `cat`, `echo`, a pipe and `wc`.

`cat README` prints the file (only its beginning is reproduced above), and `echo` prints its arguments. In `ls | grep c` the output of `ls` is passed to `grep`, which keeps only the lines containing the letter `c`: `cat`, `echo`, `wc`, `sync` and `console`. Finally, `wc README` prints the number of lines, words and bytes of the file: 48, 336 and 2441; the last value is the same as the size shown by `ls`. The optional kernel test suite (`usertests -q`) was not run.

4.3 Observations

(1) Three programs shipped with xv6. `ls`, `cat` and `grep` (the system also contains `echo`, `wc`, `mkdir`, `rm`, `kill`, `ln` and others, see the listing above).

(2) Features needed for a pipe. A pipe needs *processes* and *inter-process communication*. The shell has to create two processes (with `fork` and `exec`) that run `ls` and `grep` concurrently, and the kernel has to provide a communication channel between them: the `pipe` system call creates a kernel buffer with a read end and a write end, which are connected to the standard output of the first process and to the standard input of the second one.

(3) xv6 shell compared to the Linux shell. The xv6 shell supports the basic features of the Linux shell (running programs, redirection and pipes) but it is much simpler: it has no command history, tab completion, variables or scripting language.

5 PART 3 – READING THE SOURCE CODE

After quitting QEMU (`Ctrl-A`, `X`) the source tree was explored. The whole system is split into two folders, which is the central idea of how an operating system is organized:

- `kernel/` is the operating system itself, which runs in privileged mode: the scheduler, memory management, the file system and the system-call handlers;
- `user/` contains the ordinary programs that run on top of it: `ls`, `cat`, `echo`, the shell and so on.

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
```

Listing 6: Reading two user programs.

The programs are short and written in ordinary C. In `user/cat.c` the functions `read()` and `write()` are not normal library functions: they are requests to the kernel. Their declarations are in `user/user.h`, which lists all the system calls available to programs:

```
cat user/user.h         # the list of system calls available to
    programs
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```

Listing 7: System calls and the place of `sys_read`.

```
// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);

$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)
```

Listing 8: The system calls declared in `user/user.h` and the result of `grep`.

5.1 Observations

(1) System calls used by `user/cat.c`.

- `open(path, O_RDONLY)` asks the kernel to open a file for reading and to return a file descriptor;
- `read(fd, buf, n)` asks the kernel to copy up to n bytes from the file (or from the standard input) into the buffer;
- `write(1, buf, n)` asks the kernel to write the bytes to file descriptor 1, the standard output (the screen);
- `close(fd)` asks the kernel to release the file descriptor;
- `exit(status)` asks the kernel to terminate the process.

(`fprintf(2, ...)`, used for error messages, is a library function that itself ends in the `write` system call.)

(2) Where `sys_read` is implemented. In the file `kernel/sysfile.c`, on line 69 (the line of `sys_read(void)`; the function body ends at line 80). The function reads its arguments from the registers of the user program (`argaddr`, `argint`, `argfd`) and calls `fileread(f, p, n)`.

(3) `kernel/` versus `user/`. The code in `kernel/` is the privileged operating system that manages the hardware and answers the system calls, while the code in `user/` consists of ordinary programs that run on top of it and can reach the hardware only by asking the kernel through system calls.

6 PART 4 – THE FIRST XV6 PROGRAM

A new user program, `sleep`, was added. It pauses for a number of clock ticks by calling a system call that already exists in the kernel. In the xv6 version used here this system call is named `pause` (see Section 6.4), whereas the assignment text calls it `sleep`.

6.1 The Program

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));    // system call into the kernel
  exit(0);
}
```

Listing 9: `user/sleep.c`.

The program first checks that exactly one argument was given (the name of the program is `argv[0]`, so `argc` must be 2). Otherwise it prints the usage message to file descriptor 2 (the standard error) and ends with the status 1. If the argument is present, `atoi` converts the string to an integer and the number of ticks is passed to the `pause` system call, which suspends the process in the kernel; afterwards the program ends with status 0.

6.2 Adding the Program to the Build

The build system must be told about the new program: in the `Makefile`, in the `UPROGS` list, the line `$U/_sleep\` was added with the same indentation (a tab) as the other lines and with the backslash at the end:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_sleep\
    ...
```

Listing 10: The `UPROGS` list of the `Makefile`.

With this line `make` compiles `user/sleep.c` into `user/_sleep` and `mkfs` copies it into the disk image, which is why `sleep` appeared in the `ls` output of Part 2.

6.3 Testing

```
$ sleep 10
$ sleep
usage: sleep <ticks>
```

Listing 11: Testing the new program in xv6.

The call `sleep 10` pauses for 10 clock ticks and then returns to the prompt without printing anything, and `sleep` without an argument prints the usage message. A program that was written by hand was thus compiled with the cross-compiler, packed into the disk image of an OS compiled from source, and executed on emulated hardware, calling a system call of the kernel.

6.4 The Renamed System Call

The program was first compiled exactly as written in the assignment, with `sleep(atoi(argv[1]))`. The build stopped with the error `implicit declaration of function 'sleep'`. The cause is in `user/user.h`: in the current version of the official xv6 repository the system call that suspends a process for a number of ticks was renamed from `sleep` to `pause` (the change of 2025-08-19 renamed `SYS_sleep` to `SYS_pause` and `sys_sleep()` to `sys_pause()`, among other kernel functions), so the header declares `int pause(int);` and no longer declares `sleep`. Replacing the call with `pause(atoi(argv[1]))` solved the problem; the name of the *user program* stays `sleep`, as required by the assignment. (In an older copy of the repository, in which `user.h` still declares `int sleep(int);`, the call is written `sleep(...)` as in the assignment.)

7 CONCLUSION

In this laboratory work xv6 was built from source, booted under QEMU and used as a small Unix: the same commands as on Linux work, including pipes, which require both processes and a kernel communication mechanism. Reading the source code showed the basic organization of an operating system: the privileged `kernel/` and the ordinary programs of `user/`, connected only by system calls. A program such as `cat` contains almost nothing but `open`, `read`, `write` and `close`, and the answer to each of these calls can be found in the kernel, for example `sys_read` in `kernel/sysfile.c`.

Finally, the new program `sleep` was written, added to the `UPROGS` list and tested; it checks its arguments and calls a real system call of a kernel that was compiled by the student. Along the way, it turned out that the system call had been renamed from `sleep` to `pause` in the current xv6, a practical reminder that the interface between user programs and the kernel is defined by the header `user/user.h` and must be read in the version at hand. Since the whole system is only about 10 000 lines of C, xv6 is small enough to be understood completely, which makes it a good base for changing the kernel side in the following laboratory works.